3 Algoritmizace - Rekurze, Brute Force, Heuristiky, Nedeterministické algoritmy

Lodě a počítá rozpoložení

Rekurze

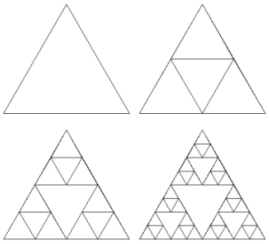
Rekurze je případ, kdy je objekt součástí sebe samotného. Můžeme si to představit jako funkci, která se provede a následně na konci zavolá samu sebe. Tím pádem se uzavře do nekonečného cyklu. Je vhodné někde ve funkci kontrolovat, jestli je nějaká podmínka splněna, v tom případě by nastal return a konec rekurzivní metody.

```
Public static void rekurze(int i){
        If(i<5){
                Systém.out.println(¨Hodnota i je: ¨ +i);
                Rekurze(i+1);
                }
}
```

Rekurze může být výhodná pro prohledávání datové struktury strom, případně třeba k hledání nejkratší cesty v grafu.

Jednoduchým příkladem využití rekurze je právě prohledávání stromu, ve kterém zavoláme rekurzivní metodu na hlavní kořen, který pro každou „Child Node“ – odnož zavolá tu samou metodu. Tím docílíme toho, že projdeme každý prvek.

Sierpinského trojúhelník Je tvořen jen pomocí trojúhelníku Takže rekurzivní objekt obsahuje svoji změnšenou kopii



rekurze v programování

Když metoda volá sama na sebe, tomu se říká rekurze.

Hlavní věc v rekurzivní metodě je příkaz, který provádí volání metody na sebe sama.

Metoda, která volá sama na sebe je “rekurzivní“.

Všechny případy rekurzí lze převést na nerekurzivní algoritmus (a naopak)

Rekurze musí obsahovat podmínku ukončení (konečnost algoritmu), jinak by byla nekonečná

To může být způsob zefektivnění rychlosti nějakých programů.

Rekurze je spojená s vysokými nároky na využití paměti zásobníku

Příklady

- strom (z větve jdou větve)
- většina her - pravidla v každém tahu jsou stejná, princip řešení je tedy stejný i když se data mění.

K čemu se dá rekurze použít?

Rekurze se dá jednoduše použít při počítání faktoriálu

Druhy

Tail recursion – rekurzní volání je posledním příkazem -> return f(). Když je rekurze optimalizována, tak na stacku nevznikají buňky volání metody, tím se ušetří místo na stacku.

Strukturální rekurze - volání funkce přes jinou funkci, nebo v jiném místě, než samostatně na konci

Backtracking - zpětné vyčíslování rekurze (obecně funkce)

Mutual (indirect) recursion - dva objekty, závisí na sobě

- funkce volající se navzájem
- objekty ukazující na sebe navzájem
- např. binární strom se skládá z uzlu a dvou stromů

Corecursion - duální k rekurzi (iterace), od jednoduchého ke složitému.

```
17      static void Main(string[] args)
18      {
19          Console.WriteLine(Faktorial(6)); //720
20          Console.WriteLine(FaktorialR(6)); //720
21      }
22
        Počet odkazů: 2
23      public static int FaktorialR(int n)
24      {
25          int vysledek;
26          if (n == 1)
27          {
28              return 1;
29          }
30          vysledek = FaktorialR(n - 1) * n;
31          return vysledek;
32      }
```

Řádek 30 začátek

N = 1, return 1 z IF

N = 2, výsledek = 1 * 2, return 2

N = 3, výsledek = 2 * 3, return 6

N = 4, výsledek = 6 * 4, return 24

N = 5, výsledek = 24 * 5, return 120

N = 6, výsledek = 120 * 6, return 720 konec řádku 30

výpis, protože se konečně executenul řádek 31

Brute Force

Bruteforce je technika procházení, která spočívá ve využití hrubé síly – tím pádem jsou postupně procházeny všechny prvky bez jakékoliv logiky. Jedná se o striktně deterministický algoritmus.

Pokud je ovšem počet dat malý, je například výhodnější než Merge Sort – třeba při sortování obyčejného pole. Při procházení například stromem jsou ovšem lepší sorty, které přestanou řešit nějakou část stromu, pokud nebude vyhovovat, jelikož bruteforce by projel i ty.

Je extrémně náročný, jak časově, tak paměťově, na druhou stranu je ale většinou velmi primitivní a vždy najde řešení (pokud je naprogramován správně)

Využívá se například na prolomení hesel typu PIN, kde je 10000 možností a víme, že jedna z nich je 100% správně, prolomení hashování a nebo již zmíněné sortování.

Heuristika

Heuristika je řešení problému, na který neznáme algoritmus, pomocí logiky. Často je takovéto řešení letmé, založené na zkušenostech a nebo intuici. Jejím hlavním cílem je najít co nejpřesnější výsledek v co nejkratším čase pro konkrétní data a proto je možné, že při jiných datech by algoritmus nefungoval.

Příkladem může být řešení rozesazení lidí do lodiček podle váhy. Bruteforce by vygeneroval stavový prostor a řešil by, kolik váží levá a pravá strana, načež by vybral nejvhodnější možnost. Monte Carlo, nedeterministický algoritmus, by někoilkrát náhodně rozsadil a poté vybran nejvhodnější možnost.

Heuristický algoritmus by nejdříve všechny podle váhy seřadil a následně je řadil postupně doleva, doprava, doleva, doprava.

Deterministické a nedeterministické algoritmy

Deterministický algoritmus je takový algoritmus, který za stejných podmínek vrátí stejný výsledek. Lze u něj předvídat budoucnost (následují krok). Třeba příklad najít číslo od 1 do 100. Tento

algoritmus půjde 1,2,3,4,5,6 (další číslo se dá předpovědět) nebo od konce 100,99,98,97 (další číslo se dá také předpovědět) nebo 10,20,30,…11,21,31 (také se dá předpovědět). Díky tomuhle platí, že každý jeho další krok je jednoznačně definován.

= každý další stav závisí na předchozím stavu

Tento algoritmus vždy za stejných vstupních podmínek vytvoří stejný výsledek. Třeba to hledání čísla od 1 do 100 (1,2,3) a číslo k najití bude 30 (vstupní parametr), tak mu to vždy bude trvat 30 pokusů.

U toho hledání čísla je naše šance na najití s počtem pokusů zvyšuje. První pokud 1/100, druhý 1/99, třetí 1/98,… = lineární šance

Nedeterministický je jeho opak, ten stejnou hodnotu nevrátí. Může mít totiž krok, který má více možností, zatímco deterministické je pevně stanovené mají. Příkladem může být například algoritmus Monte Carlo, které pracuje jenom s přibližným výsledkem, jelikož má pokaždé jiná data na vstupu. realitě, třeba hod kostkou (pořádně hodit), se nedá předpovědět, co nám padne, víme jen že to bude 1,2,3,4,5,6. Všechny tyto hodnoty mají stejnou šanci.

V počítačích je dobré zmínit, že Random number není úplně nedeterministické (generuje ho procesor, který je stroj z křemíku, nebo podle času, takže by se dalo velmi složitěpředpovědět) ale bude se u maturity brát jako nedeterministické. Třeba hádání čísla od 1 do 100, RN (random number) bude hádat jedno z těchto čísel, šance 1/100, jako druhý bude hádat taky 1 do 100 (kdyby byla kolekce ve které by byla už zkoušená čísla tak by trval moc dlouho (Constains je lineární složitosti)) a jeho šance bude 1/100 a to pořád. Takže je konstantní (1/100). Na rozdíl od deterministického nebude dávat stejný výsledek při stejných vstupních parametrech. To uhodnutí může být třeba na 1000pokus nebo na první.

Monte Carlo

Monte Carlo je algoritmus, který pracuje jenom s určitou částí dat. Pokud máme třeba 100 hodnot a máme vybrat nejnizší, Monte Carlo náhodně zvolí například 30 hodnot a z nich vrátí přibližný výsledek. Proto může být nejmenší číslo 1 a v dalším případě 50.